

# AgPlane: Programmable OS-Level IFC Policy Enforcement for AI Agent Harnesses

Paper #479  
Anonymous Submission

## ACM Reference Format:

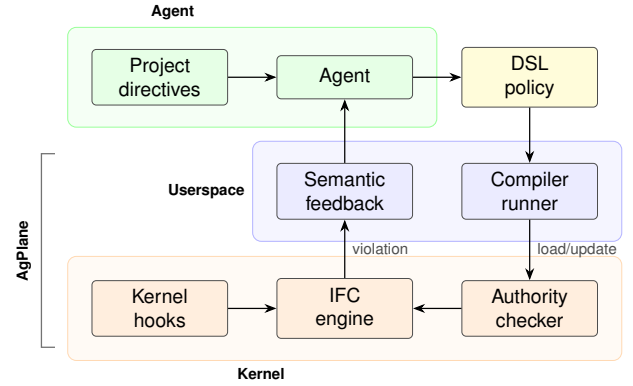
Paper #479. 2026. AgPlane: Programmable OS-Level IFC Policy Enforcement for AI Agent Harnesses. In *Proceedings of 2026 ACM SIGOPS Annual Technical Conference (ATC '26)*. ACM, New York, NY, USA, 3 pages.

AI agents are widely used for coding, DevOps, and enterprise workflows[8, 21, 23–25], increasingly in production deployments that run on real infrastructure, handle sensitive data, and interact with CI/CD pipelines. Besides the LLM, agents require AI agent harnesses: software around the model that improves agent performance while enforcing behavioral constraints for safety, security, and compliance [2].

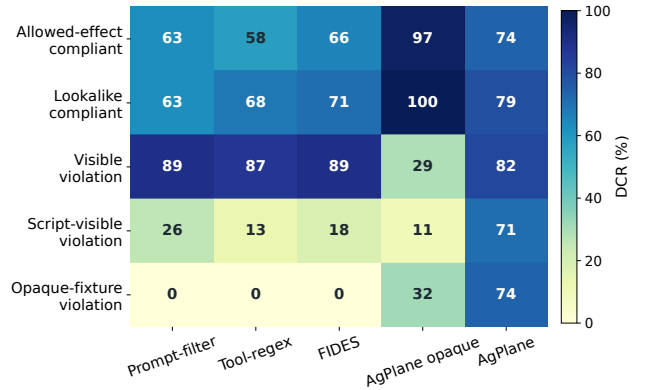
A major component of such a harness is a policy engine [19]: the part that observes and enforces constraints over the agent’s concrete actions. In practice, projects encode many such constraints as behavioral policies in instruction files (CLAUDE.md, AGENTS.md) [4, 14], so harnesses can provide these instructions to the model directly. Because LLMs comply with instructions probabilistically [12, 13, 15] and may violate them through planning errors, prompt-injection drift, and opaque tool or script behavior [8, 11, 24], harnesses also require deterministic policy engines to enforce compliance at runtime [7, 16, 20].

Many of these policies constrain system-level actions, yet existing policy engines leave a *semantic gap*: *AI agent intent and directives* are expressed in natural language, but enforcement must decide over *system actions*. Our empirical study of instruction-files in 64 highly popular projects finds that 64% statements are behavioral directives, and 83% involving system-level behavior: commands executed, files accessed, and network connections made. Existing approaches cannot close this gap. Tool-call-based policy systems [6, 7, 17, 20, 22] in agent runtimes such as Claude Code and Codex, whether LLM guardrails or regex-like checkers, lose track of indirect system actions when agents shell out or spawn subprocesses: the same `git push` can be reached through a `bash -c` wrapper, or a compiled binary created in a previous session; and at the repository level, 81% projects contain cross-event directives that single tool-level interception cannot enforce (e.g., “run tests before commit”).

OS-level enforcement with static policy is also insufficient: most directives reference project or task context that static rules cannot resolve (e.g., “never modify upstream source,” where *upstream source* must be resolved from the repository; “do not update dependencies unless requested”; “the reviewer sub-agent only writes to its project directory”). We



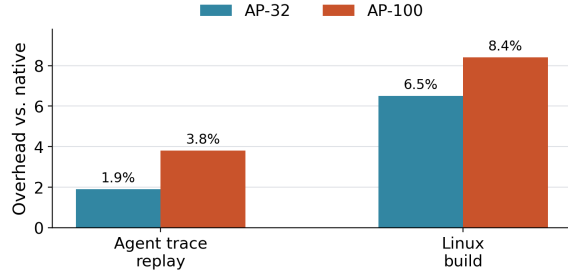
**Figure 1.** AgPlane architecture: policy generation, compilation, kernel enforcement, and feedback loop.



**Figure 2.** RQ2: Decision Compliance Rate by trace family and system. AgPlane’s advantage concentrates on indirect execution paths (script and hidden families) that tool-call baselines cannot observe.

find that 74% system-level directives require context absent from low-level OS events. OS-level enforcement systems and sandboxes for traditional software [3, 5, 9, 18] can observe all system actions but expect pre-defined static policy and return only system events without semantic context, leaving the agent unable to follow its behavioral directives.

We argue that an agent-harness policy engine should be programmable by agents and take effect at OS level: it should let agents use live project and task context to enforce actions and adapt at runtime. Our key observation is that, while

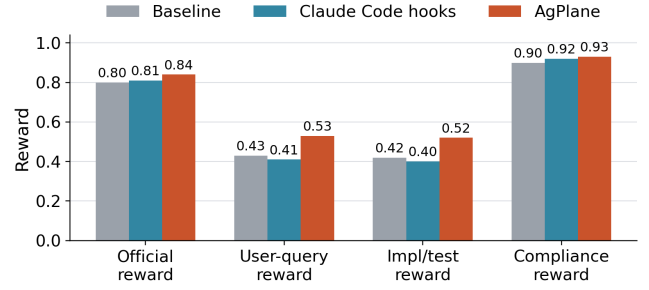


**Figure 3.** RQ3: End-to-end overhead normalized to native execution, for agent trace replay and Linux kernel build.

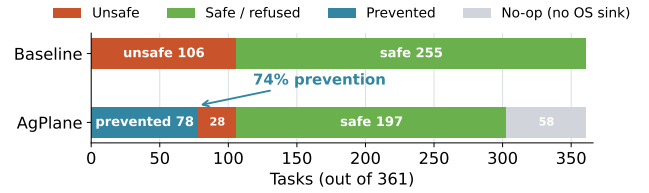
safety constraints must come from higher authority, the required context is mainly within agents closest to the task: they already read the repository, interpret the current task, and resolve abstract references into concrete commands and paths before enforcement can begin. This makes agents the natural producer of concrete policy from directives, also reflecting the fact that instruction files are increasingly maintained by agents [1, 10].

Realizing this creates three challenges. First, policy specification must be high-level enough for agents to easily generate from natural language with minimal token consumption, yet concrete enough to compile to deterministic kernel checks. Second, enforcement must track policy-relevant state across full system without imposing prohibitive per-syscall overhead. Third, since many rules are about safety and security, programmability must not become an authority bypass. Unlike traditional sandboxes threat model where a trusted administrator authors policy, here agents are both author and constrained subject, so they need an interface to add constraints without weakening inherited policy set by trusted administrators or parent agents or introducing safety risks.

AgPlane is a runtime-agnostic, programmable OS-level policy enforcement system for AI agent harnesses. Agents express policies in a compact DSL generated from natural-language; AgPlane compiles them into labeled information-flow rules and loads them into an eBPF enforcement engine across processes, files, and network kernel hooks. When matching a rule, AgPlane returns semantic feedback: which rule matched, why, and what to do next. Each rule independently selects one of three effects: notify (observe), block (pre-operation denial), or kill (process termination). A layered authority model ensures that higher-authority rules (e.g., global “do not leak secrets”) cannot be weakened by agent-authored policy. On decision-compliance benchmark, AgPlane resolves 2.0–3.2× more violations than prompt-filter, tool-regex, FIDES [6] (tool-level IFC), and feedback-free kernel IFC by covering indirect execution paths that tool-call interception cannot, while adding 1.9% end-to-end overhead on agent workloads and up to 8.4% on kernel builds. On 21



**Figure 4.** RQ4: OctoBench 21-task subset reward breakdown by system.



**Figure 5.** RQ5: Outcome distribution on 361 OpenASafety tasks. AgPlane reduces baseline-unsafe outcomes from 106 to 28 (74% prevention rate).

real coding tasks (Figure 4), a policy agent submitted 110 runtime deltas to refine rules for the task-executing sub-agent, improving task reward over baseline and tool-call hooks. On safety benchmark of 361 personal-assistant tasks, AgPlane prevents 74% of baseline-unsafe behaviors using separate agent-generated policies before unsafe execution as in higher authority path.

We make three contributions:

1. An **empirical study** of 64 projects that characterizes these gaps.
2. **AgPlane**, a programmable OS-level policy enforcement system that addresses them.
3. An **evaluation** on our decision-compliance benchmark building on empirical study, external coding-task and safety benchmarks covering workplace and personal-assistant tasks.

## References

- [1] Anthropic. 2025. Claude Code. <https://code.claude.com/docs>.
- [2] Birgitta Böckeler. 2026. Harness Engineering for Coding Agent Users. <https://martinfowler.com/articles/harness-engineering.html>. Published April 2, 2026.
- [3] Canonical Ltd. 2024. AppArmor Security Profiles. <https://apparmor.net/>.
- [4] Worawalan Chatlatanagulchai, Kundjanasith Thonglek, Brittany Reid, Yutaro Kashiwa, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, and Hajimu Iida. 2025. On the Use of Agentic Coding Manifests: An Empirical Study of Claude Code. arXiv:2509.14744. <https://arxiv.org/abs/2509.14744>

- [5] Cilium Project. 2026. Tetragon: eBPF-based Security Observability and Runtime Enforcement. <https://tetragon.io/>.
- [6] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643. <https://arxiv.org/abs/2505.23643>
- [7] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813. <https://arxiv.org/abs/2503.18813>
- [8] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., Vancouver, Canada, 82895–82920. doi:10.52202/079017-2636
- [9] Jake Edge. 2015. A seccomp overview. <https://lwn.net/Articles/656307/>.
- [10] Matthias Galster, Seyedmoein Mohsenimofidi, Jai Lal Lulla, Muhammad Auwal Abubakar, Christoph Treude, and Sebastian Baltes. 2026. Configuring Agentic AI Coding Tools: An Exploratory Study. arXiv:2602.14690. <https://arxiv.org/abs/2602.14690>
- [11] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. Association for Computing Machinery, Copenhagen, Denmark, 79–90. doi:10.1145/3605764.3623985
- [12] Yuxin Jiang, Yufei Wang, Xingshan Zeng, Wanjuan Zhong, Liangyou Li, Fei Mi, Lifeng Shang, Xin Jiang, Qun Liu, and Wei Wang. 2023. Follow-Bench: A Multi-level Fine-grained Constraints Following Benchmark for Large Language Models. arXiv:2310.20410. <https://arxiv.org/abs/2310.20410>
- [13] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. doi:10.1162/tacl\_a\_00638
- [14] Jai Lal Lulla, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M. Zhang, Sebastian Baltes, and Christoph Treude. 2026. On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents. arXiv:2601.20404. <https://arxiv.org/abs/2601.20404>
- [15] Yunjia Qi, Hao Peng, Xiaozhi Wang, Amy Xin, Youfeng Liu, Bin Xu, Lei Hou, and Juanzi Li. 2025. AGENTIF: Benchmarking Instruction Following of Large Language Models in Agentic Scenarios. arXiv:2505.16944. <https://arxiv.org/abs/2505.16944>
- [16] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Singapore, 431–445. doi:10.18653/v1/2023.emnlp-demo.40
- [17] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI Agents with Privilege Control. arXiv:2504.11703. <https://arxiv.org/abs/2504.11703>
- [18] The Linux Kernel Documentation. 2025. Landlock: Unprivileged Access Control. <https://www.kernel.org/doc/html/latest/userspace-api/landlock.html>.
- [19] Vivek Trivedy. 2026. The Anatomy of an Agent Harness. <https://www.langchain.com/blog/the-anatomy-of-an-agent-harness>. Published March 10, 2026.
- [20] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2026. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE)*. Association for Computing Machinery, New York, NY, USA, 12 pages. doi:10.1145/3744916.3764546
- [21] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations*. OpenReview.net, Singapore, 38 pages. <https://openreview.net/forum?id=OJd3ayDDoF>
- [22] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. 2025. GuardAgent: Safeguard LLM Agents via Knowledge-Enabled Reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, Vancouver, Canada, 68316–68342. <https://openreview.net/forum?id=2nBcjCZrrP>
- [23] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., Vancouver, Canada, 50528–50652. doi:10.52202/079017-1601
- [24] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 10471–10506. doi:10.18653/v1/2024.findings-acl.624
- [25] Yusheng Zheng, Yanpeng Hu, Tong Yu, and Andi Quinn. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. arXiv:2508.02736. <https://arxiv.org/abs/2508.02736>